BeaconKit 아키텍처 분석 - 06. 상태 전환 메커니즘

1. 상태 전환 개요

BeaconKit의 상태 전환은 **StateProcessor**를 통해 수행되며, 이더리움 2.0 사양을 기반으로 하되 CometBFT의 특성에 맞게 수정되었습니다. 핵심적인 차이점은 **빈 슬롯이 없다**는 것으로, 모든 CometBFT 높이가 비콘 슬롯에 1:1 대응됩니다.

1.1 StateProcessor 구조

```
type StateProcessor struct {

// logger는 정보 및 오류 로깅에 사용

logger log.Logger

// cs는 비콘 체인의 체인 사양

cs ChainSpec

// signer는 암호화 작업에 사용되는 BLS 서명자

signer crypto.BLSSigner

// fGetAddressFromPubKey는 검증자 공개키가 제안자 주소와 일치하는지 확인

fGetAddressFromPubKey  func(crypto.BLSPubkey) ([]byte, error)

// executionEngine은 트랜잭션 실행을 담당하는 엔진

executionEngine ExecutionEngine

// ds는 예치 계약에 대한 페이로드 예치 확인을 허용

ds deposit.StoreManager
```

```
// metrics는 서비스의 메트릭
metrics *stateProcessorMetrics
// logDeneb1Once는 Deneb1 포크 정보를 최대 한 번만 로깅하도록 강제
logDeneb1Once sync.Once
}
```

1.2 핵심 전환 메서드

```
func (sp *StateProcessor) Transition(
ctx ReadOnlyContext,
st *state.StateDB,
blk *ctypes.BeaconBlock,
) (transition.ValidatorUpdates, error) {
if  blk == nil {
return  nil, nil
}

// STEP 1: 다음 슬롯 처리
validatorUpdates, err := sp.ProcessSlots(st, blk.GetSlot())
if  err != nil {
return  nil, err
}
```

```
// STEP 2: 다음 블록의 포크 버전을 위한 상태 준비
logForkProcessing := ctx.VerifyPayload() && !ctx.VerifyRandao()
if cache.IsStateCachingActive(sp.cs, blk.Slot) {
logForkProcessing = ctx.VerifyPayload()
}
if err = sp.ProcessFork(st, blk.GetTimestamp(), logForkProcessing); err != nil
return nil, err
}

// STEP 3: 블록 처리
if err = sp.ProcessBlock(ctx, st, blk); err != nil {
return nil, err
}

return validatorUpdates, nil
}
```

2. 슬롯 처리 (ProcessSlots)

2.1 BeaconKit의 슬롯 처리 특징

BeaconKit은 이더리움 합의 사양의 `process_slots` 와 다릅니다:

이더리움 방식:

- 빈 슬롯 허용 (블록이 제안되지 않은 슬롯)
- 여러 슬롯을 한 번에 처리 가능
- 슬롯 101, 102가 비어있고 103에서 블록 수신 시 모든 슬롯 처리

BeaconKit 방식:

- 빈 슬롯 없음 (모든 CometBFT 높이 = 비콘 슬롯)
- 항상 다음 슬롯만 처리 (stateSlot + 1)
- 각 슬롯을 타임스탬프와 엄격히 연결

```go
func (sp *StateProcessor) ProcessSlots(
st *state.StateDB, slot math.Slot,
) (transition.ValidatorUpdates, error) {
var  res transition.ValidatorUpdates


stateSlot, err := st.GetSlot()
if  err != nil {
return  nil, err
}
// 현재 슬롯과 동일하면 처리할 것 없음
if  slot == stateSlot {
return  res, nil
}
// 정확히 다음 슬롯이어야 함 (빈 슬롯 없음)
if  slot != stateSlot+1 {
```

```
return nil, fmt.Errorf("slot %d does not match expected slot %d", slot, stateSl
}

// 슬롯 처리
if err = sp.processSlot(st); err != nil {
return nil, err
}

// 에포크 경계 처리 (슬롯이 에포크의 첫 번째 슬롯인 경우)
if slot.Unwrap()%sp.cs.SlotsPerEpoch() == 0 {
var epochUpdates transition.ValidatorUpdates
if epochUpdates, err = sp.processEpoch(st); err != nil {
return nil, err
}
res = append(res, epochUpdates...)
}

// 상태 슬롯 업데이트
if err = st.SetSlot(slot); err != nil {
return nil, err
}

return res, nil
```

```
}
```

2.2 개별 슬롯 처리 (processSlot)

```
func (sp *StateProcessor) processSlot(st *state.StateDB) error {
stateSlot, err := st.GetSlot()
if err != nil {
return err
}

// STEP 1: 이전 상태 루트 계산 및 저장
prevStateRoot := st.HashTreeRoot()
if err = st.UpdateStateRootAtIndex(
stateSlot.Unwrap()%sp.cs.SlotsPerHistoricalRoot(), prevStateRoot,
); err != nil {
return err
}

// STEP 2: 최신 블록 헤더 가져오기 (상태 루트가 없음)
latestHeader, err := st.GetLatestBlockHeader()
if err != nil {
return err
}
```

```
// STEP 3: 이전 블록의 상태 루트 설정
if (latestHeader.GetStateRoot() == common.Root{}) {
latestHeader.SetStateRoot(prevStateRoot)
if err = st.SetLatestBlockHeader(latestHeader); err != nil {
return err
}
}

// STEP 4: 블록 루트 업데이트
return st.UpdateBlockRootAtIndex(
stateSlot.Unwrap()%sp.cs.SlotsPerHistoricalRoot(),
latestHeader.HashTreeRoot(),
)
}
```

3. 블록 처리 (ProcessBlock)

3.1 블록 처리 단계

```
func (sp *StateProcessor) ProcessBlock(
ctx ReadOnlyContext,
st *state.StateDB,
```

```
blk *ctypes.BeaconBlock,
) error {
// STEP 1: 부모 블록 제안자 공개키 검색 (EL 클라이언트 알림용)
parentProposerPubkey, err := st.ParentProposerPubkey(blk.GetTimestamp())
if err != nil {
return err
}

// STEP 2: 블록 헤더 처리
if err = sp.processBlockHeader(ctx, st, blk); err != nil {
return err
}

// STEP 3: 실행 페이로드 처리
if err = sp.processExecutionPayload(ctx, st, blk, parentProposerPubkey); err !
return err
}

// STEP 4: 출금 처리
if err = sp.processWithdrawals(st, blk); err != nil {
return err
}
```

```go
// STEP 5: RANDAO 리빌 처리
if err = sp.processRandaoReveal(ctx, st, blk); err != nil {
return err
}

// STEP 6: 블록 작업 처리 (예치, 자발적 탈퇴 등)
if err = sp.processOperations(ctx, st, blk); err != nil {
return err
}

// STEP 7: 상태 루트 검증 (검증 모드인 경우)
if !ctx.VerifyResult() {
return nil
}

stateRoot := st.HashTreeRoot()
if blk.GetStateRoot() != stateRoot {
return errors.Wrapf(
ErrStateRootMismatch, "expected %s, got %s",
stateRoot, blk.GetStateRoot(),
)
}
```

```
return  nil

}
```

3.2 블록 헤더 처리

블록 헤더 처리는 다음을 포함합니다:

- 슬롯 검증
- 제안자 인덱스 검증
- 부모 루트 검증
- 상태 루트 설정 (빈 값으로, 나중에 채워짐)

3.3 실행 페이로드 처리

```
func (sp *StateProcessor) processExecutionPayload(

ctx ReadOnlyContext,

st *state.StateDB,

blk *ctypes.BeaconBlock,

parentProposerPubkey *crypto.BLSPubkey,

) error {

// 실행 엔진과의 통신을 통한 페이로드 검증

// newPayload 호출로 실행 레이어 검증

// 상태 업데이트 (최신 실행 페이로드 헤더 등)

}
```

3.4 RANDAO 처리

```go
func (sp *StateProcessor) processRandaoReveal(
ctx ReadOnlyContext,
st *state.StateDB,
blk *ctypes.BeaconBlock,
) error {
// RANDAO 리빌 서명 검증
// RANDAO 믹스 업데이트
// 랜덤성 생성에 기여
}
```

4. 에포크 처리 (processEpoch)

4.1 에포크 전환 단계

```go
func (sp *StateProcessor) processEpoch(st *state.StateDB) (transition.ValidatorU
currentEpoch, err := st.GetEpoch()
if  err != nil {
return  nil, err
}

// STEP 1: 현재 활성 검증자 추적 (변경사항 감지용)
currentActiveVals, err := getActiveVals(st, currentEpoch)
if  err != nil {
```

```
return  nil, err
}

// STEP 2: 보상 및 페널티 처리 (현재 비활성화)
// if err = sp.processRewardsAndPenalties(st); err != nil {
// return nil, err
// }

// STEP 3: 레지스트리 업데이트 (검증자 활성화/비활성화)
if  err = sp.processRegistryUpdates(st);  err != nil {
return  nil, err
}

// STEP 4: 유효 잔액 업데이트
if  err = sp.processEffectiveBalanceUpdates(st);  err != nil {
return  nil, err
}

// STEP 5: 슬래싱 리셋 (현재 비활성화)
// if err = sp.processSlashingsReset(st); err != nil {
// return nil, err
// }
```

```go
// STEP 6: RANDAO 믹스 리셋
if err = sp.processRandaoMixesReset(st); err != nil {
return nil, err
}

// STEP 7: 검증자 세트 캡 적용 (BeaconKit 특화)
if err = sp.processValidatorSetCap(st); err != nil {
return nil, err
}

// STEP 8: 검증자 세트 변경사항 계산
return sp.computeValidatorSetDiff(st, currentActiveVals)
}
```

4.2 검증자 레지스트리 업데이트

```go
func (sp *StateProcessor) processRegistryUpdates(st *state.StateDB) error {
// 1. 활성화 대기 중인 검증자들 처리
// 2. 탈퇴 대기 중인 검증자들 처리
// 3. 활성화 큐 관리
// 4. 탈퇴 큐 관리
// 5. 검증자 상태 전환 (PENDING_ACTIVE → ACTIVE 등)
```

```
}
```

4.3 유효 잔액 업데이트

```
func (sp *StateProcessor) processEffectiveBalanceUpdates(st *state.StateDB) erro

// 1. 모든 검증자의 실제 잔액 확인

// 2. 유효 잔액 계산 (32 ETH 단위로 반올림)

// 3. 유효 잔액이 변경된 검증자 업데이트

// 4. 총 활성 잔액 재계산

}
```

4.4 검증자 세트 캡 (BeaconKit 특화)

```
func (sp *StateProcessor) processValidatorSetCap(st *state.StateDB) error {

// BeaconKit의 커스텀 기능

// 활성 검증자 수를 특정 한도로 제한

// CometBFT의 성능 특성에 맞춘 최적화

}
```

5. 포크 처리 (ProcessFork)

5.1 포크 전환 메커니즘

```go
func (sp *StateProcessor) ProcessFork(
st *state.StateDB,
timestamp math.U64,
logForkProcessing  bool,
) error {
// 타임스탬프 기반 포크 버전 결정
forkVersion := sp.cs.ActiveForkVersionForTimestamp(timestamp)
// 상태의 현재 포크 버전과 비교
currentFork, err := st.GetFork()
if  err != nil {
return  err
}
// 포크 전환이 필요한 경우 처리
if !version.Equals(currentFork.CurrentVersion, forkVersion) {
return  sp.transitionToFork(st, forkVersion, logForkProcessing)
}
return  nil
}
```

5.2 지원되는 포크들

Deneb Fork:

- 블롭 트랜잭션 지원

- KZG 커밋먼트 및 증명
- 데이터 가용성 레이어

Electra Fork:

- 실행 요청 (Execution Requests)
- 검증자 예치/출금/통합 요청
- 향상된 스테이킹 기능

6. 작업 처리 (processOperations)

6.1 블록 작업 유형

```
func (sp *StateProcessor) processOperations(
ctx ReadOnlyContext,
st *state.StateDB,
blk *ctypes.BeaconBlock,
) error {
body := blk.GetBody()
// STEP 1: 예치 처리
if err := sp.processDeposits(ctx, st, body.GetDeposits()); err != nil {
return err
}
// STEP 2: 자발적 탈퇴 처리
if err := sp.processVoluntaryExits(ctx, st, body.GetVoluntaryExits()); err !=
return err
```

```
}
// STEP 3: BLS 주소 변경 처리
if err := sp.processBLSToExecutionChanges(ctx, st, body.GetBLSToExecutionChange
return err
}
// STEP 4: 실행 요청 처리 (Electra+)
if body.GetExecutionRequests() != nil {
if err := sp.processExecutionRequests(ctx, st, body.GetExecutionRequests()); e
return err
}
}
return nil
}
```

6.2 예치 처리

```
func (sp *StateProcessor) processDeposits(
ctx ReadOnlyContext,
st *state.StateDB,
deposits []*ctypes.Deposit,
) error {
for _, deposit := range deposits {
```

```
// 1. 예치 서명 검증

// 2. 예치 데이터 유효성 검사

// 3. 새 검증자 추가 또는 기존 검증자 잔액 증가

// 4. 검증자 상태 업데이트

}

}
```

7. 상태 검증 및 최적화

7.1 컨텍스트 기반 검증

```
type ReadOnlyContext interface {

VerifyPayload() bool  // 실행 페이로드 검증 여부

VerifyRandao() bool  // RANDAO 검증 여부

VerifyResult() bool  // 최종 결과 검증 여부

MeterGas() bool  // 가스 측정 여부

}
```

검증 모드별 동작:

- **블록 빌딩**: 모든 검증 비활성화 (성능 우선)
- **ProcessProposal**: 모든 검증 활성화 (안전성 우선)
- **FinalizeBlock**: 페이로드 검증만 활성화 (이미 검증된 블록)

7.2 성능 최적화

상태 루트 계산 스킵:

```
if !ctx.VerifyResult() {
return nil // 상태 루트 계산 생략
}
```

가스 측정 조건부 활성화:

```
txCtx := transition.NewTransitionCtx(ctx, consensusTime, proposerAddress).
WithMeterGas(isCacheActive) // 캐시 활성화 시에만 가스 측정
```

8. 오류 처리 및 검증

8.1 상태 일관성 검증

```
// 상태 루트 불일치 검증
if blk.GetStateRoot() != stateRoot {
return errors.Wrapf(
ErrStateRootMismatch, "expected %s, got %s",
stateRoot, blk.GetStateRoot(),
)
}

// 슬롯 순서 검증
```

```
if  slot != stateSlot+1 {
return  fmt.Errorf("slot %d does not match expected slot %d", slot, stateSlot+1)
}
```

8.2 암호학적 검증

```
// BLS 서명 검증
func (sp *StateProcessor) GetSignatureVerifierFn(
st *state.StateDB,
) (func(*ctypes.BeaconBlock, crypto.BLSSignature) error, error) {
// 검증자 공개키 조회
// 도메인 계산
// BLS 서명 검증 함수 반환
}
```

9. 메트릭 및 모니터링

9.1 성능 메트릭

```
type stateProcessorMetrics struct {
slotProcessingTime prometheus.Histogram
blockProcessingTime prometheus.Histogram
epochProcessingTime prometheus.Histogram
validatorUpdatesCount prometheus.Counter
```

```
}
```

9.2 상태 전환 추적

- 슬롯 처리 시간
- 블록 처리 시간
- 에포크 처리 시간
- 검증자 업데이트 수
- 포크 전환 이벤트

10. 설계 원칙

10.1 결정론적 실행

- 모든 노드에서 동일한 상태 전환
- 비결정적 요소 제거
- 재현 가능한 계산

10.2 점진적 검증

- 단계별 검증으로 조기 실패 감지
- 불필요한 계산 방지
- 성능과 안전성의 균형

10.3 포크 호환성

- 타임스탬프 기반 포크 전환

- 하위 호환성 유지
- 점진적 기능 활성화

이 상태 전환 메커니즘은 BeaconKit이 이더리움 호환성을 유지하면서도 CometBFT의 특성에 최적화된 방식으로 블록체인 상태를 관리하는 핵심 시스템입니다.